

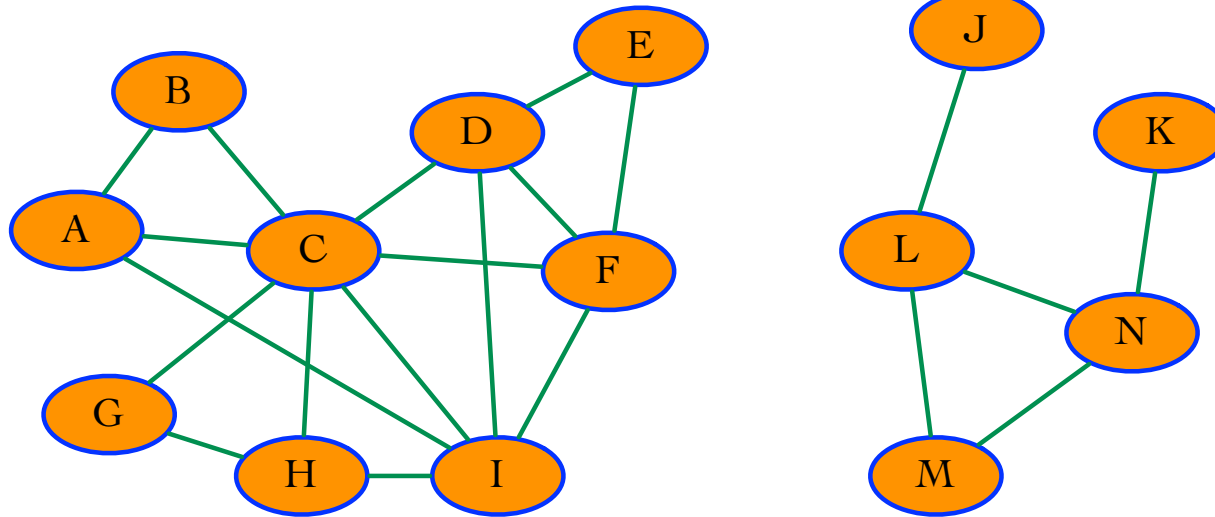
Divide and Conquer

CSC 3449

Rodrigo Canaan

Adapted with permission from Theresa t

Path



Definition

A *walk* between two vertices is a sequence of vertices with the property that each consecutive pair in the sequence is connected by an edge.

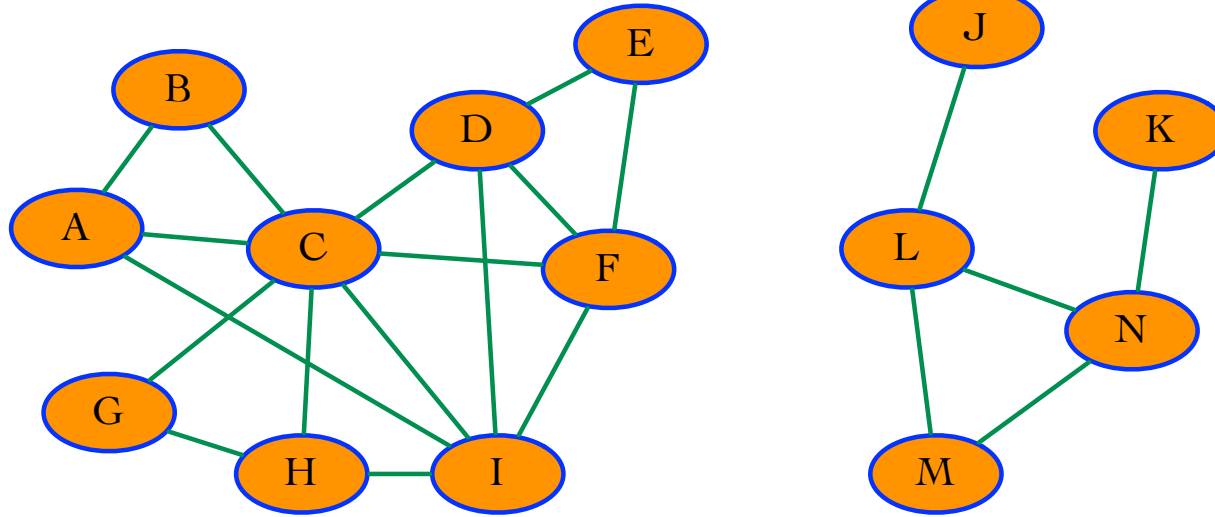
There are many walk connecting A and E.

One of these is A, C,D, E, another is A, B, C, G,H, I , F, E.

Note that a walk may contain repetition, so A, **C**, D, F, **C**, I is a walk

A,D, E is not a walk connecting A and E.

Path



Definition

A *path* between two vertices is a sequence of unique vertices with the property that each consecutive pair in the sequence is connected by an edge.

Equivalently, a *path* is a walk with no repetition of vertices

Note on naming conventions

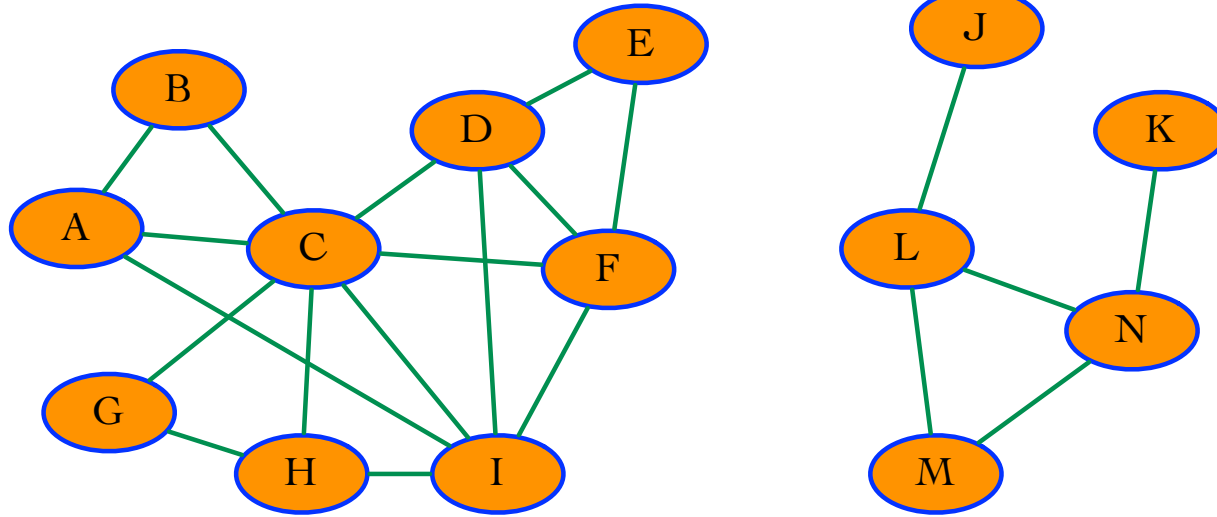
Intuitively, we defined:

- A walk: any route through the graph (may repeat vertices)
- A path: a route with no repetitions.

Some books use a different convention:

- A path: any route through the graph (what we're calling a walk)
- A simple path: a path with no repetitions.

Reachability



Definition

A vertex u is reachable from a vertex v if there is a path from v to u .

A is reachable from B, E, I among others.

A is not reachable from L

Reachability

Reachability - Problem Statement

Input: A graph $G = \{V, E\}$ and a vertex v

Output: A list of all vertices reachable from v

Intuition: A vertex u is reachable from v if:

- A. $u = v$, in other words, every vertex is reachable from itself
- B. u is a neighbor of v
- C. u is a neighbor of some vertex w , and w is reachable from v

You can also verify that case B is a special case of C

Reachability

Reachability - Problem Statement

Input: A graph $G = \{V, E\}$ and a vertex v

Output: A list of all vertices reachable from v

This observation suggests a recursive algorithm almost directly:

- Flag v as reachable
- Recursively repeat this for all of v 's neighbors

We only need to take care to avoid repeating the process on a vertex already flagged as reachable

Reachability

Reachability - Problem Statement

Input: A graph $G = \{V, E\}$ and a vertex v

Output: A list of all vertices reachable from v

DFS-visit (G, v)

Input: A graph $G = \{V, E\}$, a vertex v . Initially, every vertex has discovered set to False

Output: Every vertex reachable from v is flagged as discovered

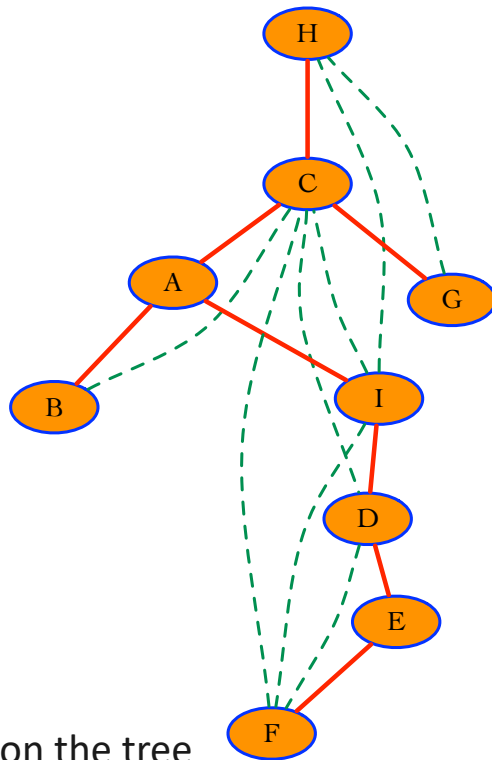
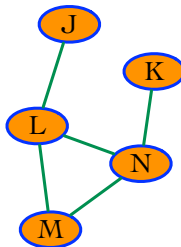
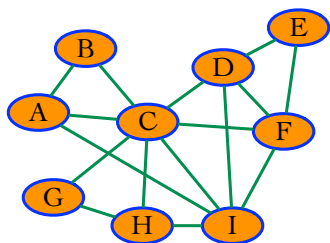
1. $v.\text{discovered} \leftarrow \text{True}$
2. For all neighbors u of v :
 - 2.1. if $u.\text{discovered}$ is False
 - 2.1.1. DFS-Visit(G, u)

Note: DFS-Visit is sometimes called DFS-Tree

Reachability

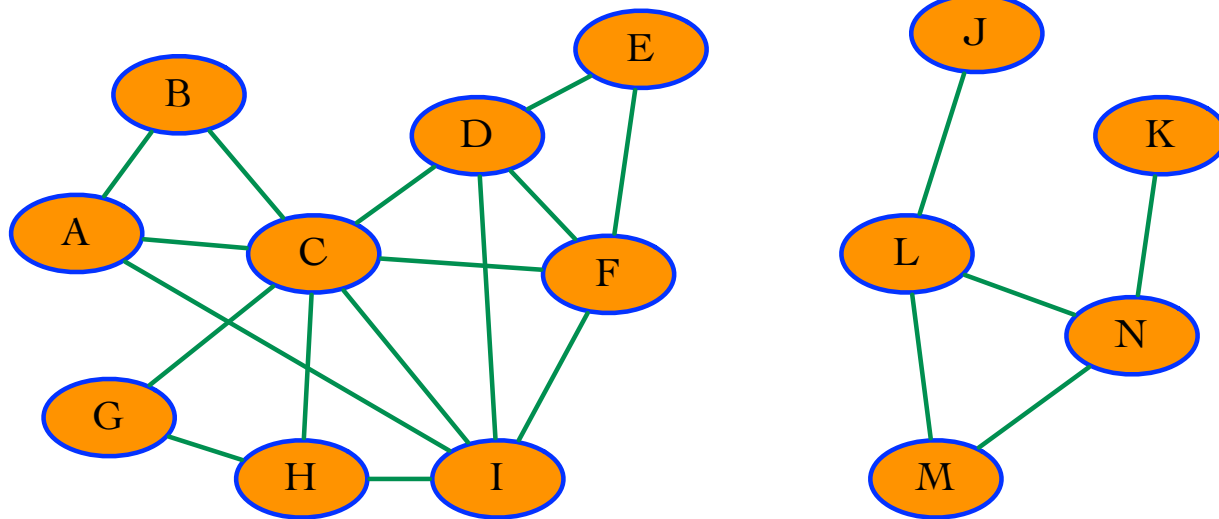
Example: DFS-Visit(Graph, H)

The search induces a tree on the graph



- We call the solid lines **Tree Edges**
 - These correspond to recursive calls
- We call the dashed lines **Back Edges**
 - These are not part of the tree
 - They correspond to edges to already visited neighbors
 - They connect a vertex to an ancestor (not the parent) on the tree

Connected Components



Definition

A connected component (or just component) of a graph is a maximal subset of vertices such that every vertex in the subset has a path to every other vertex in the subset.

There are two components: $\{A, B, C, D, E, F, G, H, I\}$ and $\{J, K, L, M, N\}$. Note that $\{L, M, N\}$ alone is not a component.

Depth-First Search

What if we want to visit all connected components of a graph?

DFS(G)

Input: A graph $G = (V, E)$.

Output: A forest of connected components of G .

1. for all $v \in V$: $\text{discovered}(v) = \text{false}$
2. for all $v \in V$:
 - 2.1. if $\text{discovered}(v) = \text{false}$
 - 2.1.1. DFS-Visit(G, v)

- Is the algorithm correct?
- What is the running time?

Running Time for DFS

DFS(G)

Input: A graph $G = (V, E)$.

Output: A forest of connected components of G .

1. for all $v \in V$: discovered(v) = false
2. for all $v \in V$:
 - 2.1. if discovered(v) = false
 - 2.1.1. DFS-Visit(G, v)

- Step 1 takes $|V|$ time.
- DFS-Visit(:) is called $|V|$ times (once per vertex).
- In DFS-Visit each edge is examined twice: $2|E|$.
- **Time complexity:** $2|V| + 2|E| = O(|V| + |E|) = O(n+m)$

Note: DFS is sometimes called DFS-Forest

Keeping Track of Additional Information

We can modify DSF-Visit to keep track of pre- and post-visit numbers

DFS-visit (G, v)

Input: A graph $G = \{V, E\}$, a vertex v . Initially, every vertex has discovered set to False, **global clock set initially to 1**

Output: Every vertex reachable from v is flagged as discovered. Vertices' pre- and post-visit

1. $v.\text{previsit} \leftarrow \text{clock}$
2. $\text{clock} \leftarrow \text{clock}+1$
3. $v.\text{discovered} \leftarrow \text{True}$
4. For all neighbors u of v :
 - 4.1. if $u.\text{discovered}$ is False
 - 4.1.1. DFS-Visit(G, u)
5. $v.\text{postvisit} \leftarrow \text{clock}$
6. $\text{clock} \leftarrow \text{clock}+1$

Keeping Track of Additional Information

Interpretation

We can think of a run of DFS-Visit as a series of “events”:

- The first time we visit each vertex is an “event”
 - Alternatively: when we start a new instance of DFS-Visit
- The moment we are done exploring all of a vertex neighbors is an event
 - Alternatively: just before we return from an instance of DFS-Visit

As we will see later, these can be useful for detecting cycles, detecting ancestor/descendant relationships, counting the number of elements in a component or subtree, among other uses

Keeping Track of Additional Information

We can similarly modify DFS to explicitly number components

DFS(G)

Input: A graph $G = (V, E)$, and a **global component counter initially set to zero**

Output: A forest of **numbered** connected components of G .

1. for all $v \in V$: $discovered(v) = \text{false}$, $component(v) = \text{None}$
2. for all $v \in V$:
 - 2.1. if $discovered(v) = \text{false}$
 - 2.1.1. DFS-Visit(G, v , **component**)
 - 2.1.2. $component = \text{component} + 1$

DFS-visit (G, v)

Input: A graph $G = \{V, E\}$, a vertex v . Initially, every vertex has discovered set to False, **and a component number**

Output: Every vertex reachable from v is flagged as discovered **and is given the same component number**

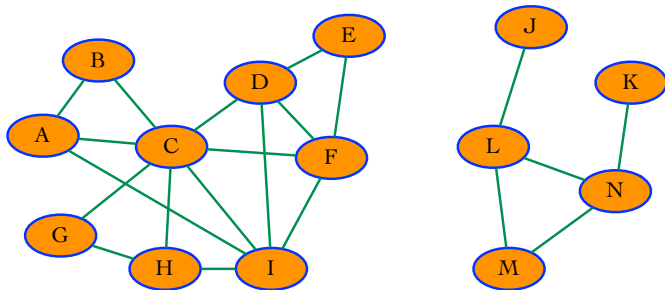
1. $v.component \leftarrow component$
2. $v.discovered \leftarrow \text{True}$
3. For all neighbors u of v :
 - 3.1. if $u.discovered$ is False
 - 3.1.1. DFS-Visit(G, u , **component**)

In other words, every time DFS “restarts” marks the beginning of a new component

Cycles and Trees

Definition

A *cycle* in an undirected graph is a walk with three or more edges, that starts and ends on the same vertex, but otherwise all vertices are distinct.



A-B-C-A is a cycle. A-B-C-D-E-F-I-A is also a simple cycle.

A-B-C-D-F-C is **not** a cycle (doesn't start and end with the same vertex)

C-D-F-C-B-A-C **not** a cycle (repeats vertex C). It can be called a **closed walk**

A-B-A is **not** a cycle (fewer than 3 edges)

Cycles and Trees

Definition

A *tree* in an undirected graph is a connected graph (or a connected component) with no cycles

We call an acyclic graph with multiple components a *forest*

Equivalent definitions (proof: exercise)

- A tree is a connected graph with n vertices and $n+1$ edges
- A tree is a connected graph where adding any edge creates a cycle
- A tree is a connected graph where removing any edge disconnects the graph
- A tree is a connected graph with a unique path between any two vertices

Cycles and Trees

Definition

A *spanning tree* of a connected graph is a subgraph with the same vertices, but only enough edges to maintain connectivity (if there are n vertices, every spanning tree has $n-1$ edges)

If a graph has cycles, it will have multiple spanning trees

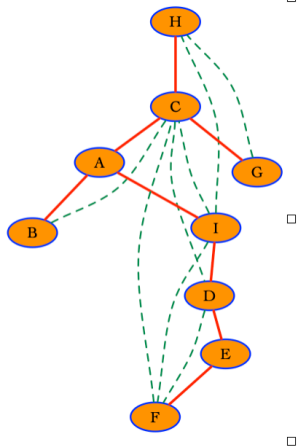
DFS induces a spanning tree in the graph

Moreover, it is a *rooted tree* (a tree with a special vertex called the root).

The choice of root also introduces a preferred direction (“down” or away from the root) along with notions like parents, children, siblings, descendants, ancestors.

In DFS, a back edge is always first discovered by a descendant, leading to an ancestor.

We call a vertex v *along* with all its descendants in the tree a *subtree rooted at v*

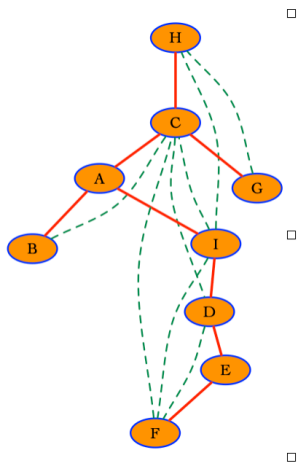


Cycles and Trees

Definition

A *spanning tree* of a connected graph is a subgraph with the same vertices, but only enough edges to maintain connectivity (if there are n vertices, every spanning tree has $n-1$ edges)

If a graph has cycles, it will have multiple spanning trees



If (u, v) is a back edge, then the path along the tree from v to u , along with the back edge, forms a *fundamental cycle*.

Different spanning trees of the same graph will result in different fundamental cycles.

However, the number of fundamental cycles of a connected graph is always $m - (n - 1)$, that is, the number of edges it has in excess of one of its spanning trees.

Summary of undirected graphs

Definitions

- *Neighbors*: two vertices that share an edge.
- *Path*: a sequence of distinct vertices where every consecutive pair are neighbors
- *Reachability*: two vertices are mutually reachable if there is a between them
- *Connected*: a graph where every pair of vertices is mutually reachable
- *Connected component*: a maximal connected subset of a graph
- *Cycle*: a path where the first and last vertices are the same, but otherwise all vertices are distinct.
- *Tree*: a connected graph with no cycles
- *Spanning tree*: a subset of a connected graph with all its vertices but only $n-1$ edges
- *Rooted tree*: a tree where a vertex is chosen as root.

DFS can be used to determine reachability, identify connected components, construct a spanning tree and identify cycles (among other uses)